

CAHIER DES CHARGES

Application Mobile de Gestion Locative

BailControl · Bailleur Edition

Projet	BailControl — Application de gestion locative
Client	
Version	1.0.0
Date	
Frontend	Flutter (iOS & Android)
Backend	
Base de données	SQLite (local) / PostgreSQL (cloud)
Statut	

— Document confidentiel · Usage privé —

1. Contexte et Objectif

1.1 Contexte

Les bailleurs (propriétaires de logements) font face à des difficultés récurrentes dans la gestion quotidienne de leurs biens locatifs :

- Suivi manuel et fastidieux des paiements de loyers
- Gestion des échéances sans outil centralisé
- Relances inefficaces des locataires en retard
- Calcul et application des pénalités de retard
- Absence de tableau de bord synthétique

1.2 Objectif du projet

Développer une application mobile privée, destinée à un usage personnel, permettant au bailleur de :

- Gérer l'ensemble de ses locataires en un seul endroit
- Suivre en temps réel l'état des paiements
- Automatiser l'envoi de rappels (SMS, WhatsApp, appels vocaux IA)
- Appliquer automatiquement des pénalités en cas de retard
- Visualiser un tableau de bord complet de sa situation locative

1.3 Périmètre

Utilisateur cible : Bailleur unique (propriétaire privé). L'application n'est pas destinée à une diffusion publique.

Plateforme : Mobile (Android & iOS) via Flutter. Backend centralisé via Django.

2. Architecture Technique

2.1 Vue d'ensemble

L'application suit une architecture client-serveur REST. Le mobile (Flutter) consomme une API JSON fournie par le backend Django.

Couche	Technologie	Rôle
Frontend	Flutter 3.x (Dart)	Interface mobile iOS & Android
Backend	Django 5.x + DRF	API REST, logique métier, tâches planifiées
Base de données	SQLite (dev) / PostgreSQL (prod)	Persistance des données
Authentification	Django SimpleJWT	Tokens JWT pour l'accès à l'API
Tâches async	Celery + Redis	Rappels automatiques planifiés
SMS / WhatsApp	Twilio / Africa's Talking API	Envoi de rappels J-5
Appels vocaux	Twilio Voice / Vonage	Appels automatiques J-1
Déploiement	Railway / Render / VPS Ubuntu	Hébergement du backend Django

2.2 Django — Détail du backend

Le backend est construit avec Django 5.x et Django REST Framework (DRF). Il expose une API RESTful consommée par l'application Flutter.

2.2.1 Applications Django (apps)

Le projet Django est structuré en plusieurs applications :

App Django	Responsabilité
accounts	Authentification bailleur, gestion du PIN, JWT
locataires	CRUD locataires, statuts, logique de transition
paiements	Enregistrement paiements, historique, marquage
rappels	Planification et envoi SMS/WhatsApp/Appels
penalites	Calcul automatique, accumulation journalière
dashboard	Agrégats statistiques, KPIs, alertes

2.2.2 Endpoints API REST (DRF)

Tous les endpoints sont préfixés par `/api/v1/` et protégés par JWT (sauf `/api/auth/`).

Méth.	Endpoint	Description
POST	/api/auth/token/	Obtenir les tokens JWT (access + refresh)
POST	/api/auth/token/refresh/	Rafraîchir le token d'accès
GET	/api/v1/locataires/	Liste de tous les locataires du bailleur
POST	/api/v1/locataires/	Créer un nouveau locataire
GET	/api/v1/locataires/{id}/	Détail d'un locataire
PUT	/api/v1/locataires/{id}/	Modifier un locataire
DEL	/api/v1/locataires/{id}/	Supprimer un locataire
PATCH	/api/v1/locataires/{id}/statut/	Changer le statut (payé, discussion, etc.)
GET	/api/v1/paiements/	Historique de tous les paiements
POST	/api/v1/paiements/	Enregistrer un paiement
GET	/api/v1/paiements/?locataire={id}	Historique d'un locataire spécifique
GET	/api/v1/penalites/	Liste des pénalités actives
POST	/api/v1/penalites/calculer/	Forcer le calcul des pénalités manuellement
GET	/api/v1/dashboard/	KPIs globaux (totaux, alertes, revenus)
GET	/api/v1/rappels/	Liste des rappels envoyés
POST	/api/v1/rappels/tester/	Tester l'envoi d'un rappel manuellement

2.2.3 Tâches planifiées (Celery Beat)

Les automatismes de rappel et de calcul de pénalités sont gérés par Celery Beat, un scheduler intégré à Django via la librairie django-celery-beat.

Tâche Celery	Fréquence	Action
verifier_echeances_j5	Quotidien 08h00	Envoyer SMS/WhatsApp J-5 aux non-payés
verifier_echeances_j1	Quotidien 09h00	Déclencher appel vocal IA J-1 aux non-payés
calculer_penalites	Quotidien 00h01	Passer en pénalité + cumuler montant journalier
nettoyer_rappels	Hebdomadaire	Archiver les rappels anciens de plus de 90 jours

2.2.4 Structure du projet Django

Arborescence recommandée du projet backend :

```

bailcontrol_backend/
├── manage.py
├── config/
│   ├── settings/
│   │   ├── base.py           # Settings communs
│   │   ├── dev.py            # SQLite, DEBUG=True
│   │   └── prod.py           # PostgreSQL, ALLOWED_HOSTS

```

```
|   ├── urls.py           # Routage principal
|   └── celery.py         # Config Celery
└── apps/
    ├── accounts/        # Authentification + JWT
    ├── locataires/      # Modèle Locataire + API
    ├── paiements/       # Modèle Paiement + API
    ├── rappels/         # Tâches Celery + services SMS
    ├── penalites/       # Calcul automatique
    └── dashboard/       # Agrégats
└── requirements.txt
└── docker-compose.yml
```

3. Modèles de Données Django

3.1 Modèle Bailleur (extension de AbstractUser)

```
class Bailleur(AbstractUser):
    pin_securite = models.CharField(max_length=6)
    telephone = models.CharField(max_length=20)
    penalite_default = models.DecimalField(max_digits=10,
decimal_places=2, default=3000)
    date_creation = models.DateTimeField(auto_now_add=True)
```

3.2 Modèle Locataire

```
class Locataire(models.Model):
    STATUT_CHOICES = [
        ("paye", "Payé"),
        ("non_paye", "Non payé"),
        ("discussion", "En discussion"),
        ("penalite", "En pénalité"),
    ]
    bailleur = models.ForeignKey(Bailleur,
on_delete=models.CASCADE)
    nom = models.CharField(max_length=150)
    telephone = models.CharField(max_length=20)
    montant_loyer = models.DecimalField(max_digits=12,
decimal_places=2)
    jour_echeance = models.IntegerField() # Jour du mois (1-31)
    statut = models.CharField(max_length=20,
choices=STATUT_CHOICES, default="non_paye")
    date_entree = models.DateField()
    penalite_journaliere = models.DecimalField(max_digits=10,
decimal_places=2)
    notes = models.TextField(blank=True)
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)
```

3.3 Modèle Paiement

```
class Paiement(models.Model):
    MODE_CHOICES = [("mobile_money", "Mobile Money"), ("especes", "Espèces"), ("virement", "Virement")]
    locataire = models.ForeignKey(Locataire,
on_delete=models.CASCADE, related_name="paiements")
    date_paiement = models.DateTimeField(auto_now_add=True)
    montant_paye = models.DecimalField(max_digits=12,
decimal_places=2)
    mois_concerne = models.DateField()
```

```

mode_paiement      = models.CharField(max_length=30,
choices=MODE_CHOICES)
remarque           = models.TextField(blank=True)

```

3.4 Modèle Rappel

```

class Rappel(models.Model):
    TYPE_CHOICES =
[("sms", "SMS"), ("whatsapp", "WhatsApp"), ("appel", "Appel vocal IA")]
    STATUT_CHOICES =
[("envoye", "Envoyé"), ("echec", "Échec"), ("en_attente", "En attente")]
    locataire       = models.ForeignKey(Locataire,
on_delete=models.CASCADE)
    type_rappel     = models.CharField(max_length=20,
choices=TYPE_CHOICES)
    date_envoi      = models.DateTimeField(auto_now_add=True)
    statut_envoi    = models.CharField(max_length=20,
choices=STATUT_CHOICES)
    message         = models.TextField()
    reponse_api     = models.JSONField(null=True)      # Réponse
brute Twilio/AT

```

3.5 Modèle Pénalité

```

class Penalite(models.Model):
    STATUT_CHOICES = [("active", "Active"), ("cloturee", "Clôturée")]
    locataire       = models.ForeignKey(Locataire,
on_delete=models.CASCADE)
    date_debut      = models.DateField()
    date_fin        = models.DateField(null=True)
    montant_journalier = models.DecimalField(max_digits=10,
decimal_places=2)
    montant_total   = models.DecimalField(max_digits=12,
decimal_places=2, default=0)
    nb_jours_retard = models.IntegerField(default=0)
    statut          = models.CharField(max_length=20,
choices=STATUT_CHOICES, default="active")

```

4. Fonctionnalités Principales

4.1 Gestion des locataires

Le bailleur peut gérer ses locataires via l'interface Flutter qui consomme l'API Django :

- Ajouter un locataire (POST /api/v1/locataires/)
- Modifier ses informations (PUT /api/v1/locataires/{id}/)
- Supprimer un locataire (DELETE /api/v1/locataires/{id}/)
- Lister tous les locataires avec filtre par statut
- Voir le détail complet : fiche, paiements, rappels, pénalités

4.2 Gestion des paiements

- Marquer un locataire comme "Payé" → POST /api/v1/paiements/
- Le statut du locataire passe automatiquement à "payé"
- Historique complet accessible via GET /api/v1/paiements/?locataire={id}
- Filtrage par mois, par mode de paiement

4.3 Système de statuts

Chaque locataire possède un statut géré par Django, avec des transitions automatiques :

Statut	Condition déclencheur	Conséquences
Payé	Bailleur enregistre un paiement	Aucun rappel, aucune pénalité
Non payé	Défaut / début de mois	Rappels J-5 et J-1 actifs
En discussion	Bailleur marque manuellement	Pas d'appel agressif, SMS uniquement
En pénalité	Date dépassée + non payé (auto)	Pénalité journalière accumulée

4.4 Automatisation des rappels (Celery)

Les rappels sont déclenchés par des tâches Celery planifiées. Django vérifie chaque matin les échéances à venir.

Rappel J-5 — SMS / WhatsApp

- Tâche : `verifier_echeances_j5` (exécutée à 08h00 chaque matin)
- Condition : `date_echeance - date_aujourd'hui == 5 ET statut != "paye"`
- Message : "Votre loyer arrive à échéance dans 5 jours. En cas de difficulté, contactez le bailleur."
- Canal : Twilio SMS API ou WhatsApp Business API

Rappel J-1 — Appel vocal IA

- | |
|---|
| → Tâche : <code>verifier_echeances_j1</code> (exécutée à 09h00 chaque matin) |
| → Condition : <code>date_echeance - date_aujourd'hui == 1</code> ET <code>statut != "paye"</code> |
| → Exception : <code>statut == "discussion"</code> → pas d'appel, SMS doux uniquement |
| → Message : "Votre loyer est dû demain. Merci de régulariser votre situation." |
| → Canal : Twilio Programmable Voice / Vonage Voice API |

4.5 Gestion des pénalités

La tâche Celery `calculer_penalites` s'exécute chaque nuit à minuit et applique la logique suivante :

- Si `date_echeance < date_aujourd'hui` ET `statut != "paye"` → statut passe à "penalite"
- Une entrée Penalite est créée ou mise à jour (`nb_jours_retard++`, `montant_total += montant_journalier`)
- Quand le bailleur enregistre le paiement → `statut = "paye"`, `penalite.statut = "cloturee"`
- Le `montant_journalier` est configurable par locataire (hérité du bailleur par défaut)

4.6 Tableau de bord

L'endpoint GET `/api/v1/dashboard/` retourne un objet JSON agrégé calculé côté Django :

```
{
  "total_locataires": 12,
  "loyers_payes": 8,
  "loyers_en_attente": 2,
  "en_penalite": 2,
  "en_discussion": 1,
  "revenus_mois": 540000,
  "penalites_dues": 36000,
  "echeances_5_jours": ["Alain N.", "Rose A.", "Pierre M."],
  "alertes": [...]
}
```

5. Sécurité

5.1 Authentification JWT

Django SimpleJWT est utilisé pour sécuriser l'API. Le flux d'authentification est le suivant :

- Le bailleur entre son PIN sur l'application Flutter
- Flutter envoie une requête POST /api/auth/token/ avec username + password (PIN haché)
- Django retourne un access_token (durée : 60 min) et un refresh_token (durée : 7 jours)
- Chaque requête API inclut le header : Authorization: Bearer <access_token>
- Le refresh token permet de renouveler l'accès sans saisir le PIN à nouveau

5.2 Protections Django

- CORS configuré avec django-cors-headers (whitelist des origines autorisées)
- Rate limiting sur les endpoints sensibles (django-ratelimit)
- Données chiffrées en transit (HTTPS obligatoire en production)
- Validation stricte des entrées via DRF Serializers
- Protection CSRF désactivée pour l'API REST (remplacée par JWT)
- Variables sensibles (SECRET_KEY, DB credentials) via variables d'environnement (.env)

5.3 Isolation des données

- Chaque queryset est filtré par bailleur connecté :
Locataire.objects.filter(bailleur=request.user)
- Impossible pour un bailleur d'accéder aux données d'un autre
- Les tokens Twilio sont stockés en variables d'environnement, jamais en base

6. Règles Métier

#	Règle	Implémentation Django
R1	Un locataire payé ne reçoit aucun rappel	Filtre : <code>.exclude(statut="paye")</code> dans les tâches Celery
R2	Un locataire en discussion n'est pas appelé	Filtre : <code>.exclude(statut__in=["paye","discussion"])</code> pour appels
R3	Pénalité automatique après échéance	Tâche <code>calculer_penalites</code> vérifie date chaque nuit
R4	Pénalité clôturée dès paiement	Signal <code>post_save</code> sur <code>Paiement</code> ferme la <code>Penalite</code> active
R5	Rappels anti-spam (1 par jour max)	Vérification <code>Rappel.objects.filter(date=today)</code> avant envoi
R6	Données isolées par bailleur	Filtre systématique <code>bailleur=request.user</code> sur tous les <code>querysets</code>
R7	Le montant pénalité est configurable	Champ <code>penalite_journaliere</code> sur <code>Locataire</code> , hérité du <code>Bailleur</code>

7. Contraintes Techniques

7.1 Hors ligne (Flutter)

- L'application Flutter gère un cache local (SQLite via sqflite) pour fonctionner sans réseau
- Les données sont synchronisées avec l'API Django dès la reconnexion
- Les rappels automatiques nécessitent que le backend Django soit en ligne

7.2 Performance Django

- Pagination obligatoire sur les endpoints liste (PageNumberPagination, 20 items/page)
- Utilisation de `select_related` et `prefetch_related` pour éviter les requêtes N+1
- Cache Redis pour le endpoint `/api/v1/dashboard/` (TTL : 5 minutes)

7.3 Déploiement

- Django servi par Gunicorn derrière Nginx en production
- Celery Worker et Celery Beat en processus séparés
- Redis comme broker Celery et backend de cache
- Fichiers statiques servis par WhiteNoise

8. Dépendances Python (requirements.txt)

Package	Version	Rôle
Django	>=5.0	Framework web principal
djangorestframework	>=3.15	Construction de l'API REST
djangorestframework-simplejwt	>=5.3	Authentification JWT
django-cors-headers	>=4.3	Gestion des origines CORS
celery	>=5.3	Tâches asynchrones planifiées
django-celery-beat	>=2.6	Scheduler Celery intégré à Django
redis	>=5.0	Broker Celery + cache
django-redis	>=5.4	Cache Django via Redis
twilio	>=9.0	Envoi SMS, WhatsApp, appels vocaux
psycopg2-binary	>=2.9	Driver PostgreSQL (production)
gunicorn	>=22.0	Serveur WSGI de production
whitenoise	>=6.7	Fichiers statiques en production
python-decouple	>=3.8	Variables d'environnement (.env)
Pillow	>=10.0	Traitement images (reçus PDF futurs)
drf-yasg	>=1.21	Documentation Swagger/OpenAPI auto

9. Évolutions Possibles

Fonctionnalité	Priorité	Notes techniques
Génération de reçus PDF	Haute	ReportLab ou WeasyPrint côté Django
Notifications push	Haute	Firebase FCM via django-fcm-notifications
Multi-bailleurs	Moyenne	Déjà préparé par le modèle Bailleur
Sauvegarde cloud	Moyenne	Django Storages + AWS S3 / Cloudinary
Statistiques avancées	Basse	Django ORM agrégats + charts Flutter
API WhatsApp Business	Haute	Meta Cloud API ou Twilio WhatsApp
Export Excel	Basse	openpyxl ou django-import-export

10. Critères de Réussite

Critère	Indicateur de validation
Application simple à utiliser	Navigation fluide, PIN < 2 secondes
Données fiables	Tests unitaires Django (coverage > 80%)
Rappels automatiques fonctionnels	SMS/Appel reçu dans les 30min après déclenchement Celery
Pénalités correctement calculées	Montant = nb_jours × montant_journalier, vérifiable
API sécurisée	Accès refusé (401) sans token JWT valide
Performance	Réponse API < 300ms pour tous les endpoints liste

Conclusion

BailControl est une application conçue pour simplifier la gestion locative d'un bailleur privé. L'architecture Django + Flutter offre une séparation claire des responsabilités : le backend Django centralise toute la logique métier, les règles de gestion des pénalités et l'automatisation des rappels via Celery, tandis que Flutter fournit une interface mobile moderne et réactive.

Le choix de Django REST Framework garantit une API robuste, documentée (Swagger via drf-yasg) et facilement extensible pour les évolutions futures : multi-bailleurs, génération de PDF, notifications push.

— Fin du document —